Estructuras de Datos

2026
Primer Cuatrimestre

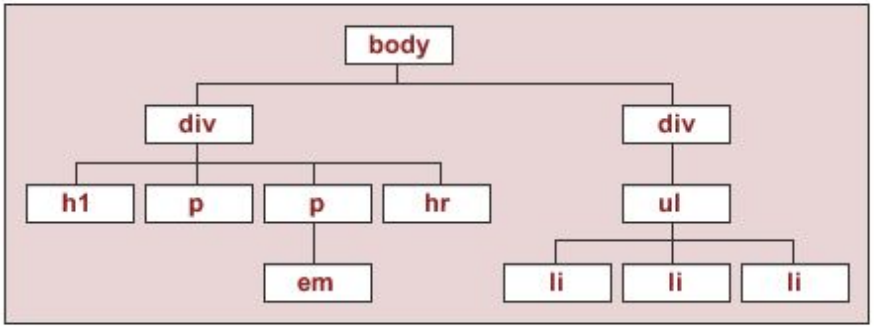
REPASO

```
<body>

  <div id="content">
    <h1>Heading here</h1>
    <p>Lorem ipsum dolor sit amet.</p>
    <p>Lorem ipsum dolor <em>sit</em> amet.</p>
    <hr>
  </div>

  <div id="nav">
    <ul>
      <li>item 1</li>
      <li>item 2</li>
      <li>item 3</li>
    </ul>
  </div>

</body>
```



TDA ÁRBOL

- Un árbol es un TDA que almacena los elementos llamados nodos jerárquicamente.
- Con la excepción del nodo raíz, cada nodo en un árbol tiene un nodo padre y cero o más nodos hijos.

TDA ÁRBOL - Interface

```
public interface Tree<E> extends Iterable<E>
{
public int size();
public boolean isEmpty();
public Iterator<E> iterator();
public Iterable<Position<E>> positions();
public E replace(Position<E> v, E e)
public Position<E> root()
public Position<E> parent(Position<E> v)
public Iterable<Position<E>> children(Position<E> v)
public boolean isInternal(Position<E> v)
public boolean isExternal(Position<E> v)
public boolean isRoot(Position<E> v)
public void createRoot(E e)
public Position<E> addFirstChild(Position<E> p, E e)
public Position<E> addLastChild(Position<E> p, E e)
public Position<E> addBefore(Position<E> p, Position<E> rb, E e)
public Position<E> addAfter (Position<E> p, Position<E> lb, E e)
public void removeExternalNode (Position<E> p)
public void removeInternalNode (Position<E> p)
public void removeNode (Position<E> p)
```

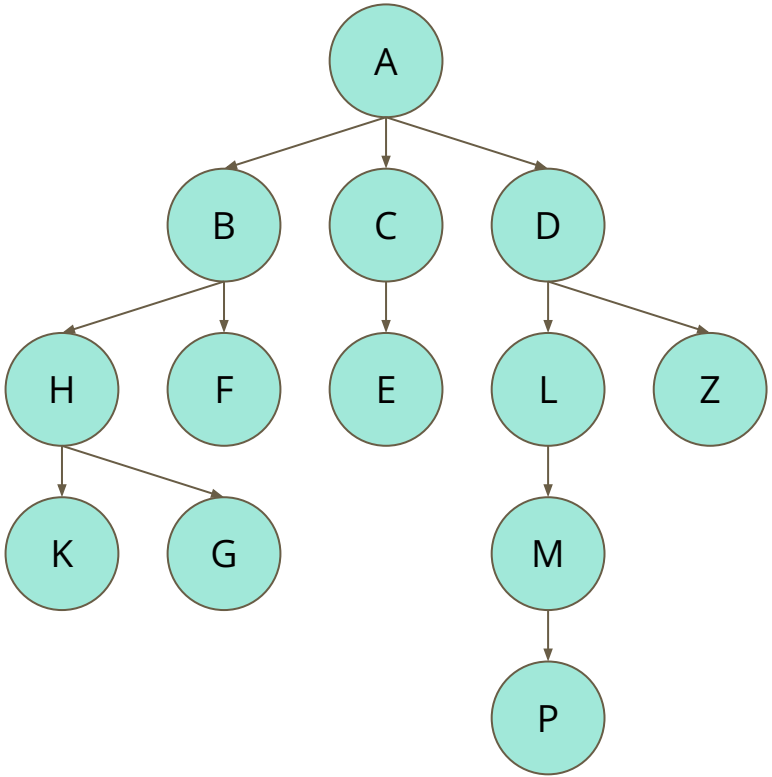
TDA ÁRBOL - Recorridos

Un recorrido de un árbol T es una manera sistemática de visitar todos los nodos de T

La literatura describe 4 formas de recorrer un árbol:

- Preorden (orden previo)
- Postorden (orden posterior)
- Inorder (orden simétrico)
- Por Niveles (level ordering)

¿Orden de ejecución de cada uno?



Resumen de los recorridos

Planteo Preorden:

CB: Si e es hoja: pre(e) = [e]

CR: pre(nodo(e, [t1 t2 ... tk])) = [e] + pre(t1) + pre(t2) + ... + pre(tk)

Planteo Postorden:

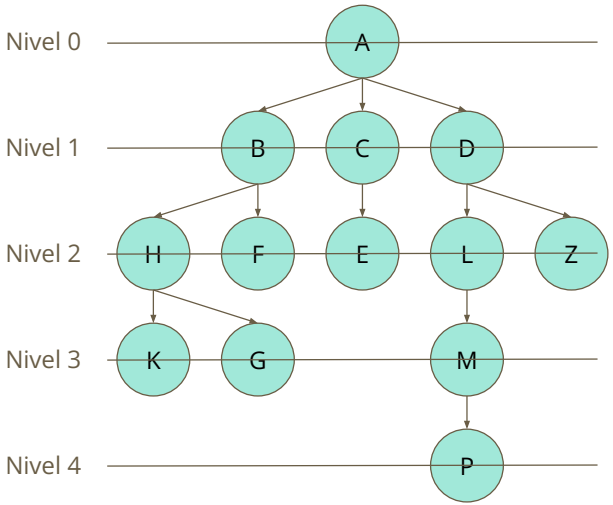
CB: Si e es hoja: post(e)

CR: post(nodo(e, [t1 t2 ... tk])) = post(t1) + post(t2) + ... + post(tk) + [e]

Planteo Inorder:

CB: Si e es hoja: in(e) = [e]

CR: in(nodo(e, [t1 t2 ... tk])) = in(t1) + [e] + in(t2) + ... + in(tk)



Búsqueda en un árbol: recorrido no exhaustivo

Buscar(t, x): buscar_rec(t, t.root(), x)

Buscar recursivo se define por casos: buscar_rec(t, p, x)

CB: rotulo(p) = x => buscar_rec(t,p,x) = p

CB: rotulo(p) ≠ x y p es hoja => buscar_rec(t,p,x) = null

CR: rotulo(p) ≠ x y p es interno =>

sean h_1, h_2, ..., h_n los hijos de p:

si $\exists h_i$: buscar_rec(t, h_i, x) ≠ null ⇒

buscar_rec(t, p, x) = buscar_rec(t, h_i, x),

si $\forall h_i$ buscar_rec(t,hi,x) = null

buscar_rec(t, p, x) = null

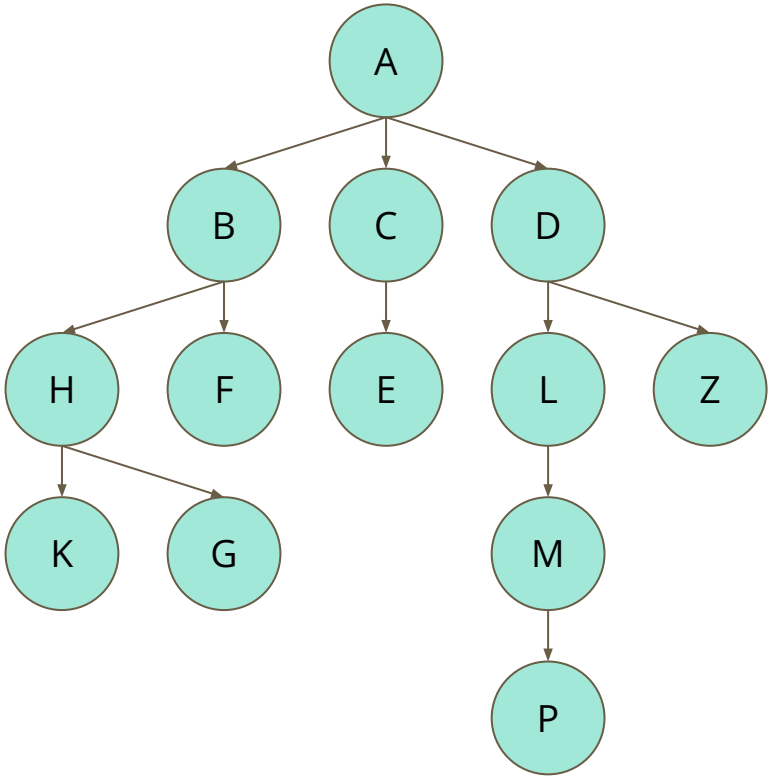
Buscar(t, x): buscar_rec(t, t.root(), x)

Algoritmo buscar_rec(t, p, x):

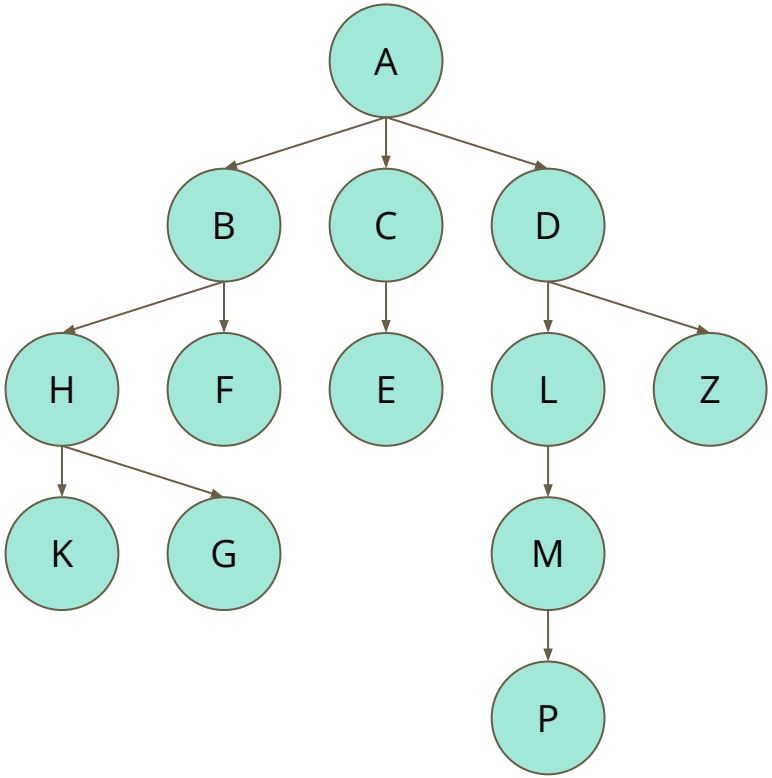
```
si rotulo(p) = x:
    retornar p
sino:
    si p es hoja: retornar null
    sino:
        para cada hijo h de p:
            res = buscar_rec(t, h, x)
            si res ≠ null: retornar res
    retornar null
```

IMPLEMENTACIONES

Un árbol



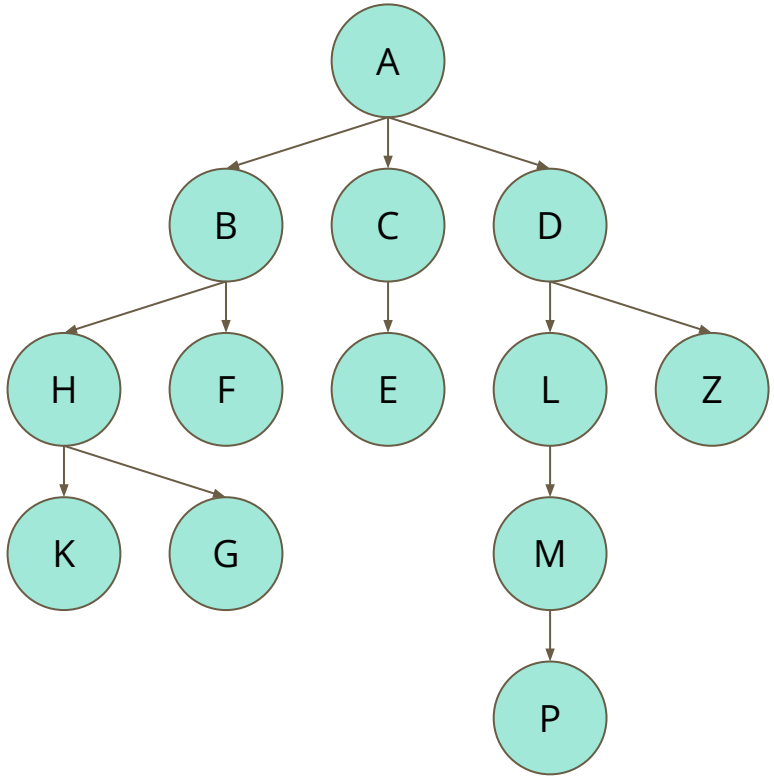
Representación de lista de hijos



Representación de lista de hijos

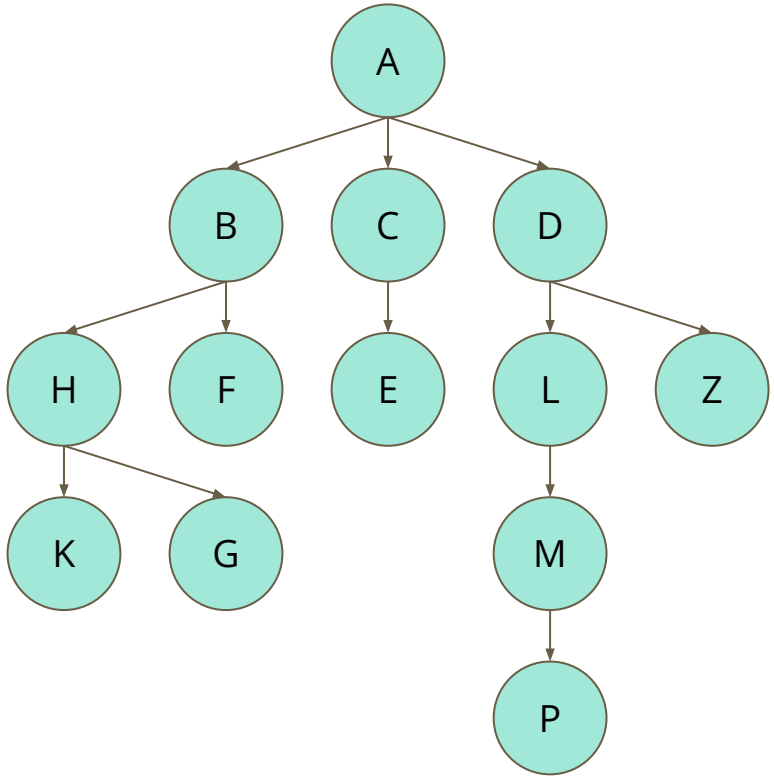
Cada nodo conoce su rótulo, o elemento, y la lista de sus nodos hijos.

El árbol conoce el nodo raíz del árbol.



Representación de lista de hijos

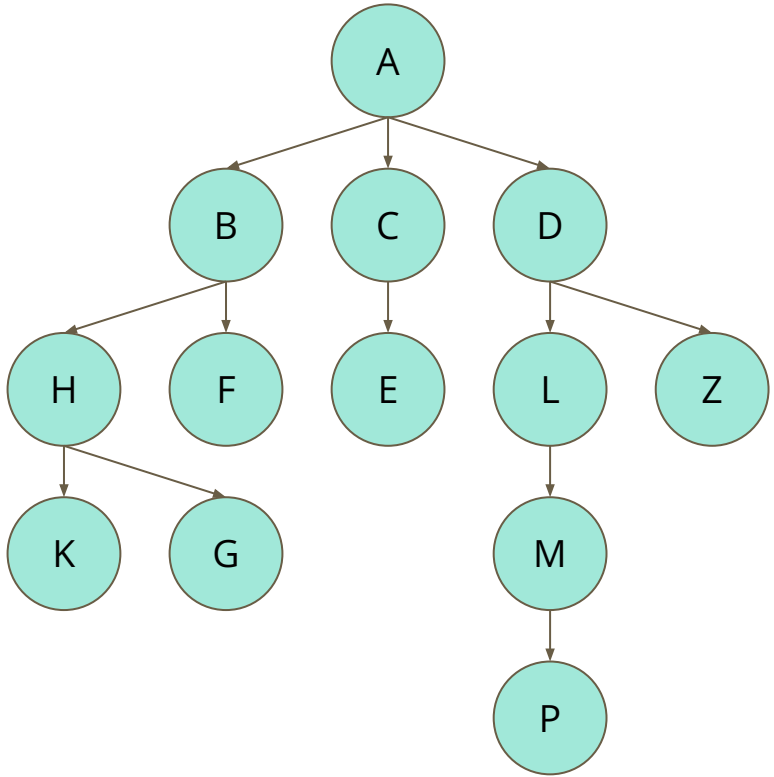
TNodo<E> implements Position<E>
elem: E hijos: PositionList<TNodo<E>>



Representación de lista de hijos

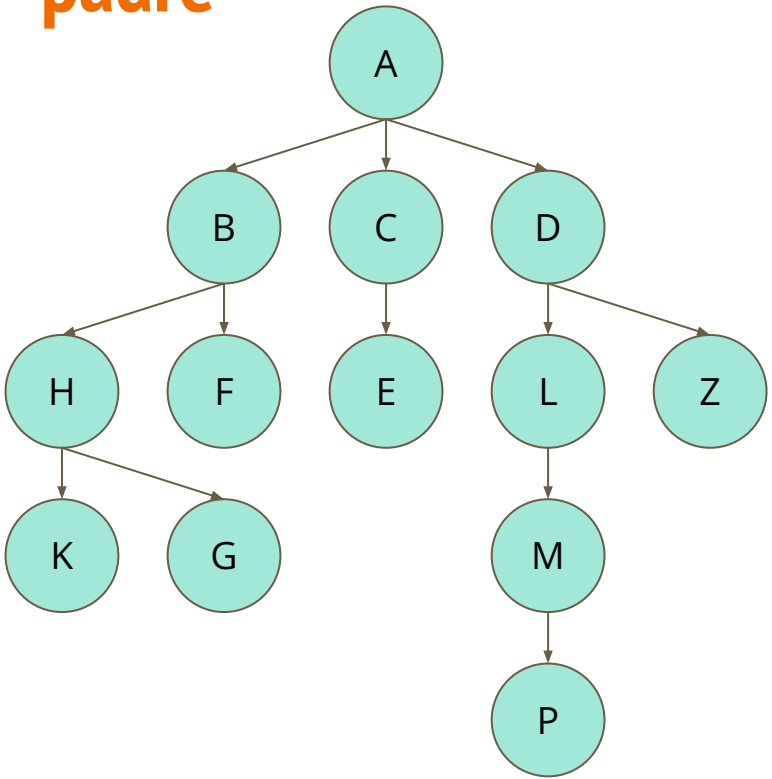
Arbol<E> implements Tree<E>
raiz: TNodo<E> size: integer

TNodo<E> implements Position<E>
elem: E hijos: PositionList<TNodo<E>>

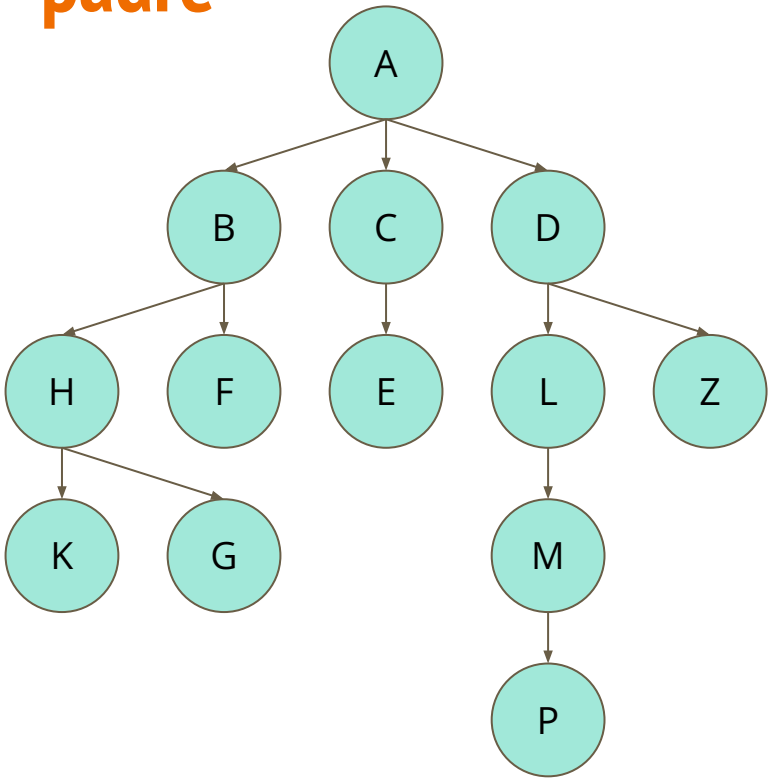


Representación de lista de hijos + padre

Esta es la propuesta del libro. El árbol conoce la raíz. Cada nodo conoce el rótulo, o elemento, la lista de nodos hijos y el nodo padre.



Representación de lista de hijos + padre

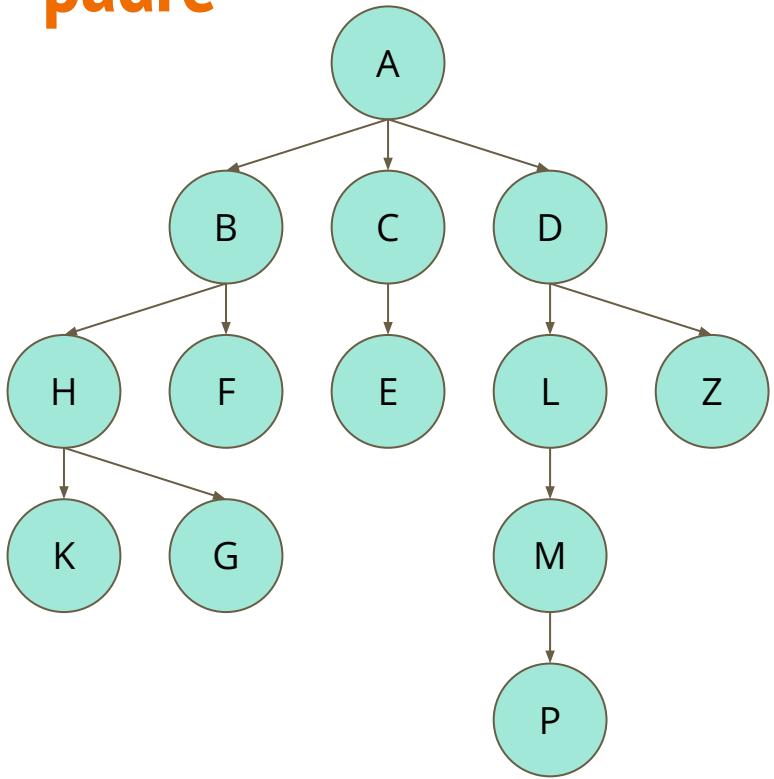


TNodo<E> implements Position<E>
elem: E padre: TNodo<E> hijos: PositionList<TNodo<E>>

Representación de lista de hijos + padre

Arbol<E> implements Tree<E>
raiz: TNodo<E> size: integer

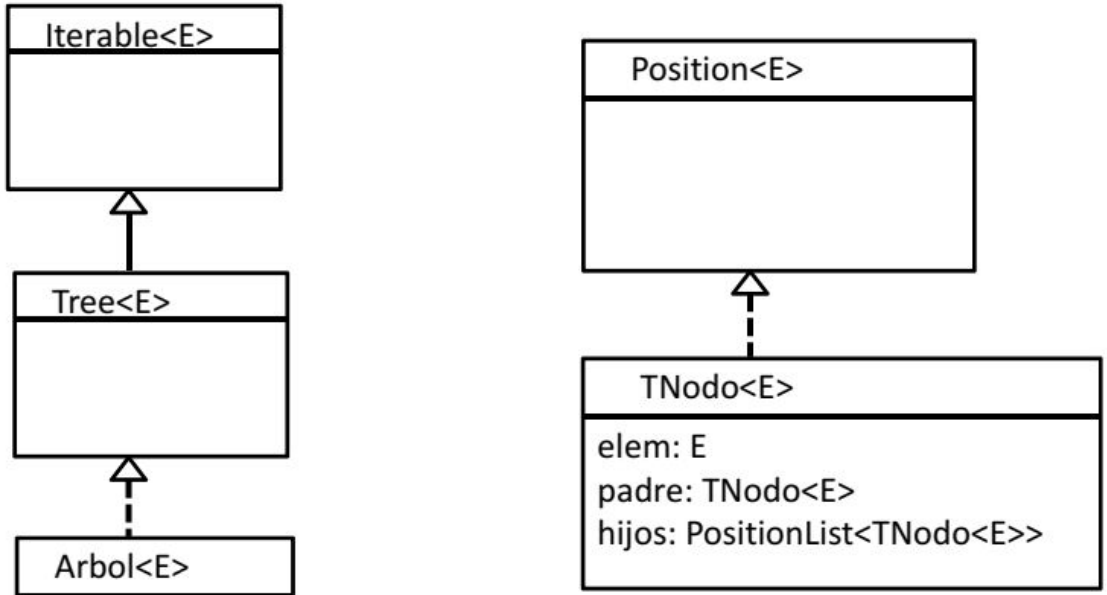
TNodo<E> implements Position<E>
elem: E padre: TNodo<E> hijos: PositionList<TNodo<E>>



Implementación de TDA Arbol

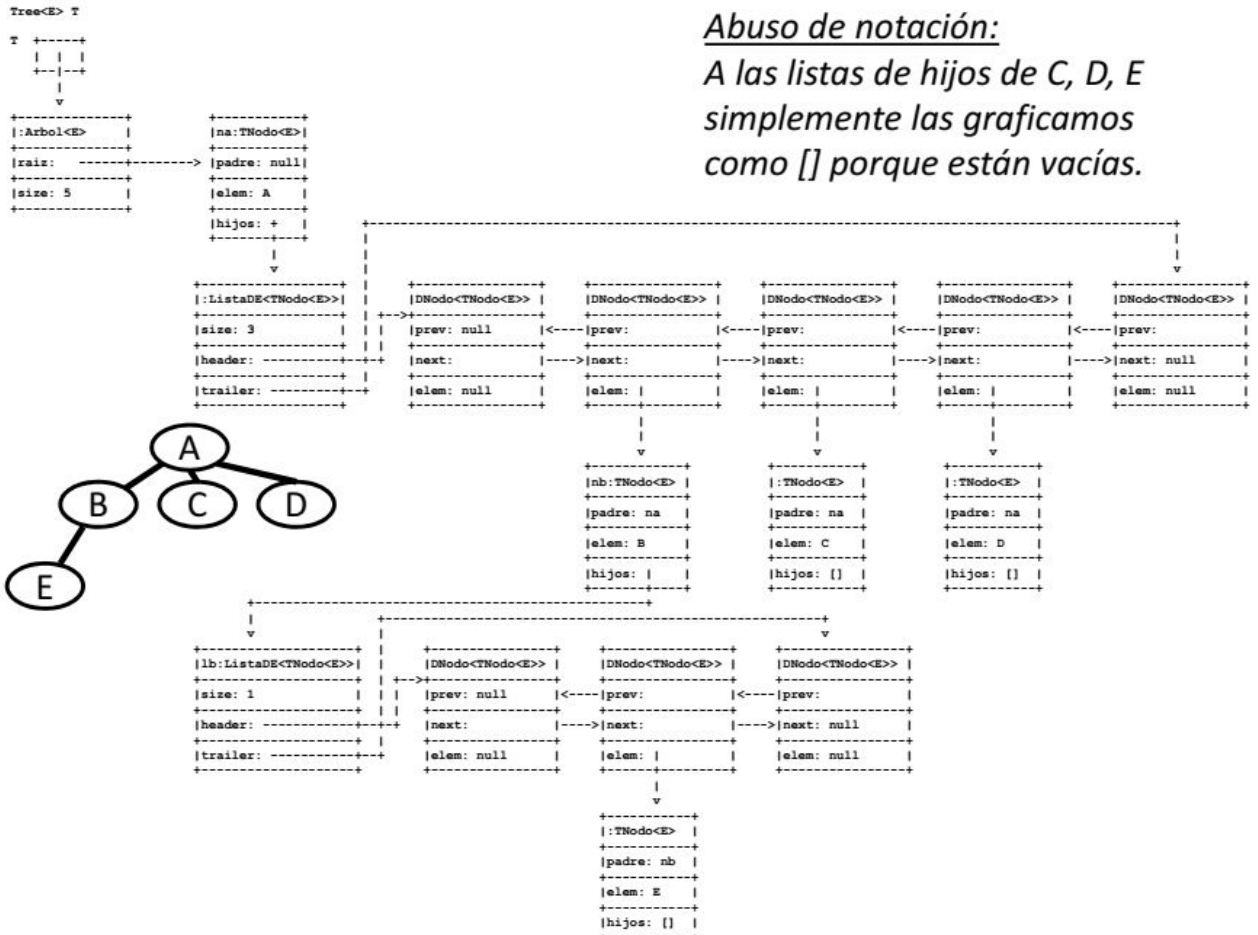
- Discutiremos la estructura de datos y la implementación de algunas de las operaciones
  - El resto de las operaciones quedan como tarea

Diagrama de clases (Representación GT)



Recordar que Iterable es una interfaz que corresponde a java.lang.Iterable, Tree y Position también son interfaces mientras que Arbol y TNodo son clases.

Diagrama de objetos (representación GT)



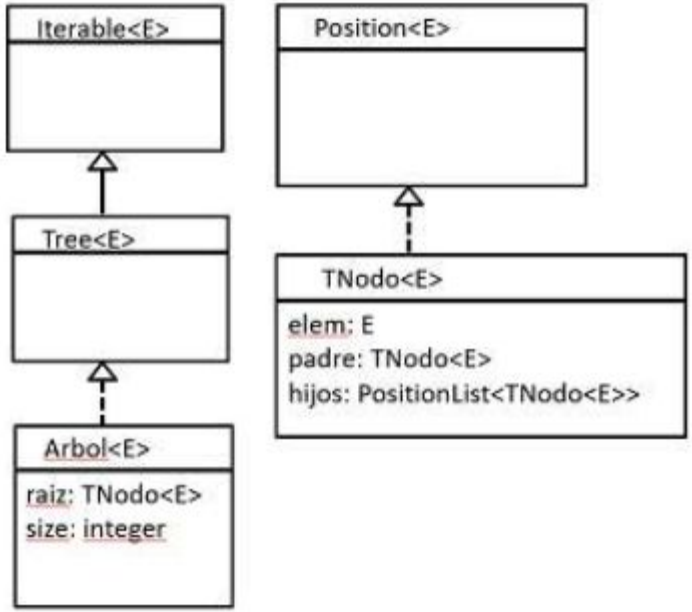
Abuso de notación:

A las listas de hijos de C, D, E simplemente las graficamos como [] porque están vacías.

Clase TNodo

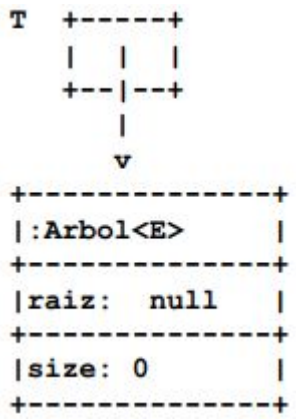
```
public class TNodo<E> implements Position<E>
{
   private E elemento;
   private TNodo<E> padre;
   private PositionList<TNodo<E>> hijos;

   public TNodo( E elemento, TNodo<E> padre ) {
      this.elemento = elemento;
      this.padre = padre;
      hijos = new DoubleLinkedList<TNodo<E>>();
   }
   public TNodo(E elemento ) { this( elemento, null); }
   public E element() { return elemento; }
   public PositionList<TNodo<E>> getHijos() { return hijos; }
   public void setElemento( E elemento ) { this.elemento = elemento; }
   public TNodo<E> getPadre() { return padre; }
   public void setPadre( TNodo<E> padre ) { this.padre = padre; }
}
```



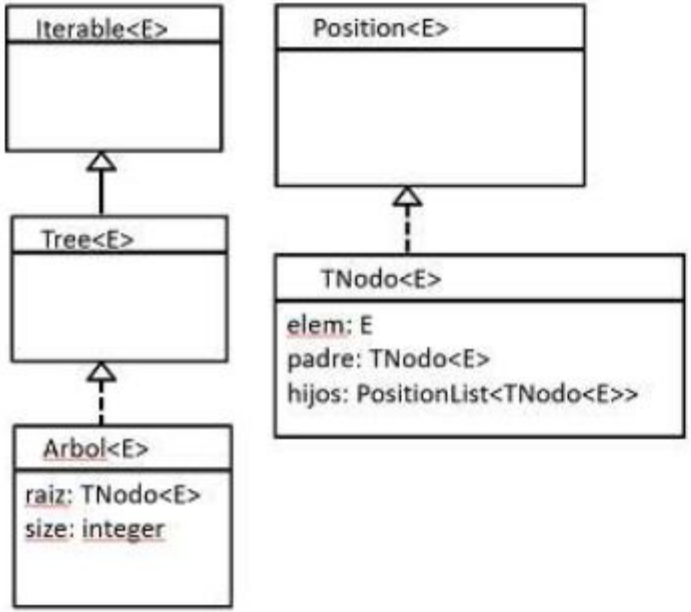
Constructor + isEmpty()

```
public class Arbol<E> implements Tree<E>
{
    protected TNodo<E> raiz;
    protected int size;
    public Arbol() {
        raiz = null;
        size = 0;
    }
    public boolean isEmpty() {
            return raiz == null;
    }
```



$T_{isEmpty}(n) \rightarrow O(1)$

root()



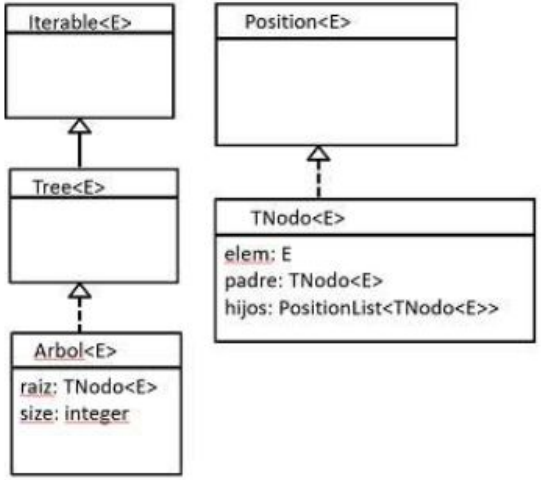

```
Tree<E> T = ...
Position<E> raiz = T.root();

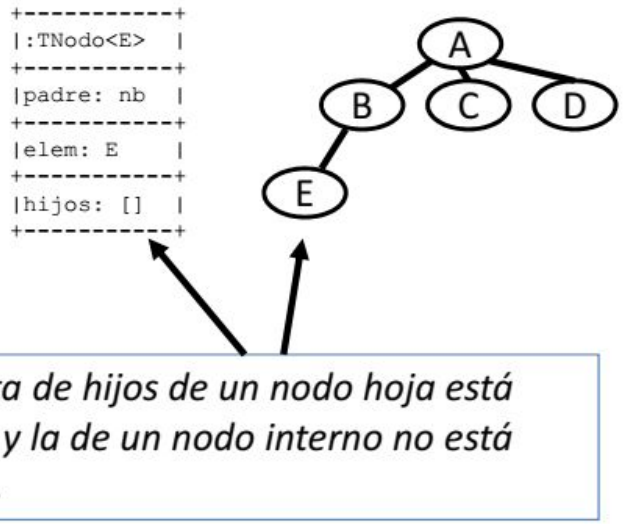
T  +-----+
   |  |  |
   +--|--+
      |
      v
+--------------+          +-----------+
|:Arbol<E>     |          |na:TNodo<E>|
+--------------+          +-----------+
|raiz:   ------+--------> |padre: null|
+--------------+          +-----------+
|size: 5       |          |elem: A    |
+--------------+          +-----------+
                          |hijos: ... |
                          +-----------+
```

```
public Position<E> root() {
    if( isEmpty() )
        throw new EmptyTreeException( “Árbol vacío” );
    return raiz;
}
```

$T_{root}(n) \rightarrow O(1)$

isExternal()

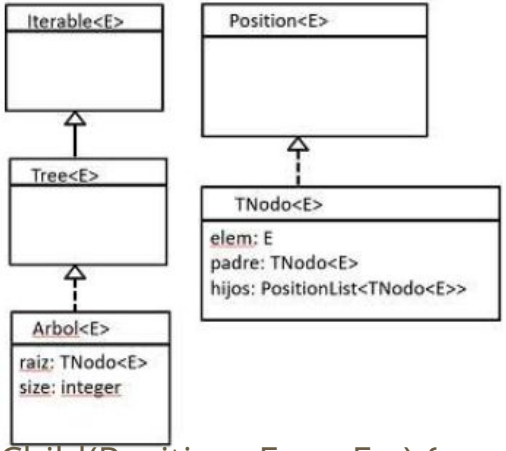


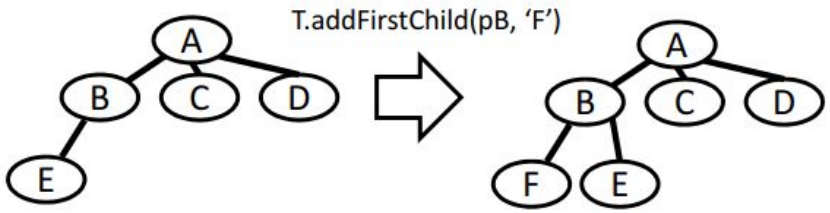



```
public boolean isExternal(Position<E> v) {
    TNodo<E> nodo = checkPosition( v );
    return nodo.getHijos().isEmpty();
}
```

$T_{isExternal}(n) \rightarrow O(1)$

addFirstChild()





Hay que castear p a TNodo para poder tener acceso a sus campos, recuperar la lista de hijos de p, insertarle un nuevo nodo del árbol con rótulo e e incrementar el tamaño del árbol.

```
public Position<E> addFirstChild(Position<E> p, E e) {
    TNodo<E> nodo = checkPosition(p);
    TNodo<E> nuevo = new TNodo<E>( e, nodo );
    nodo.getHijos().addFirst( nuevo );
    size++;
    return nuevo;
}
```

$T_{addFirstChild}(n) \rightarrow O(1)$

addBefore(p, rb, e)

Estrategia:

- Chequeos:
  - Que las posiciones p y rb sean válidas (casteen a TNodo<E>)
- Creo un TNodo nuevo con padre p y elemento e
- Tomo todos los hijos de p
  - Encuentro rb
    - ¿Qué pasa si no lo encuentro?
  - Inserto el nodo nuevo antes de p
- Aumento el tamaño del árbol
- Devuelvo el nodo creado

addBefore(p, rb, e)

```
import tdalista.Position; // Necesito usar posiciones de lista por lista de hijos
public Position<E> addBefore(Position<E> p, Position<E> rb, E e) {
    TNodo<E> n = checkPosition( p );
    TNodo<E> hd = checkPosition( rb );
    TNodo<E> nuevo = new TNodo<E>( e, n ); // Conecto el nodo nuevo con el padre









}
```

addBefore(p, rb, e)

```
import tdalista.Position; // Necesito usar posiciones de lista por lista de hijos
public Position<E> addBefore(Position<E> p, Position<E> rb, E e) {
    TNodo<E> n = checkPosition( p );
    TNodo<E> hd = checkPosition( rb );
    TNodo<E> nuevo = new TNodo<E>( e, n ); // Conecto el nodo nuevo con el padre
    PositionList<TNodo<E>> hijos = n.getHijos();
    // Buscar dónde está rb en la lista de hijos de p
    boolean encontre = false;
    tdalista.Position<TNodo<E>> pp = hijos.first();
    while( pp != null && !encontre )
        // Testeo si el elemento corriente de la lista de hijos de p es rb
        if( hd == pp.element() ) encontre = true // Sí, es! => terminé el bucle
        else pp = (pp != hijos.last() ? hijos.next(pp) : null); // No es => avanzo




}
```

addBefore(p, rb, e)

```
import tdalista.Position; // Necesito usar posiciones de lista por lista de hijos
public Position<E> addBefore(Position<E> p, Position<E> rb, E e) {
   TNodo<E> n = checkPosition( p );
   TNodo<E> hd = checkPosition( rb );
   TNodo<E> nuevo = new TNodo<E>( e, n ); // Conecto el nodo nuevo con el padre
   PositionList<TNodo<E>> hijos = n.getHijos();
   // Buscar dónde está rb en la lista de hijos de p
   boolean encontre = false;
   tdalista.Position<TNodo<E>> pp = hijos.first();
   while( pp != null && !encontre )
      // Testeo si el elemento corriente de la lista de hijos de p es rb
      if( hd == pp.element() ) encontre = true // Sí, es! => terminé el bucle
      else pp = (pp != hijos.last() ? hijos.next(pp) : null); // No es => avanzo
   if( !encontre ) // => Hay un problema con los argumentos
      throw new InvalidPositionException( "p no es padre de rb" );
   hijos.addBefore( pp, nuevo ); // Inserto al nodo nuevo delante de rb
   size++; // Incremento el tamaño del árbol
   return nuevo; // Retorno el nodo creado
}
```

$T_{addBefore}(n) \rightarrow O(\text{cantidad de hijos de p}) = O(\text{grado de p})$

removeExternalNode(p)

Estrategia: Tengo que eliminar un nodo hoja p

- Chequeos:
  - Que el árbol no esté vacío
  - Que la posición p sea válida (castee a TNodo<E>)
  - Que p sea una hoja
  - ¿Qué pasa si el árbol tiene un único nodo, su raíz?
- Actualizar:
  - Los hijos del padre de p
    - Recorro la lista de hijos y elimino a p
    - ¿Qué pasa si no lo encuentro? Por ejemplo, p ya había sido eliminado
  - Seteo a null el rótulo de p (para ayudar a checkPosition) y el padre de p
  - Cantidad de nodos del árbol

removeExternalNode(p)

```
import tdalista.Position; // Necesito usar posiciones de lista por lista de hijos
public void removeExternalNode (Position<E> p) {
    if( isEmpty() ) throw new InvalidPositionException( “Arbol vacío” );
    TNodo<E> n = checkPosition( p );
    if( !n.getHijos().isEmpty() ) throw new InvalidPositionException( “p no es hoja” );
    if ( p == root() ) { raiz = null; size = 0; n.setElemento(null); return; } // Parche por caso no considerado




}
```

removeExternalNode(p)

```
import tdalista.Position; // Necesito usar posiciones de lista por lista de hijos
public void removeExternalNode (Position<E> p) {
    if( isEmpty() ) throw new InvalidPositionException( “Arbol vacío” );
    TNodo<E> n = checkPosition( p );
    if( !n.getHijos().isEmpty() ) throw new InvalidPositionException( “p no es hoja” );
    if ( p == root() ) { raiz = null; size = 0; n.setElemento(null); return; } // Parche por caso no considerado
    TNodo<E> padre = n.getParent(); // Obtengo el padre de n
    PositionList<TNodo<E>> hijosPadre = padre.getHijos(); // Obtengo los hijos del padre de n
    boolean encontre = false; // Hago una búsqueda lineal de n en la lista de hijos del padre de   n
    tdalista.Position<TNodo<E>> pp = null; // va a almacenar la posición de n en la lista de hijos de su padre
    Iterable<tdalista.Position<TNodo<E>>> posiciones = hijosPadre.positions();
    Iterator<tdalista.Position<TNodo<E>>> it = posiciones.iterator();









}
```

removeExternalNode(p)

```
import tdalista.Position; // Necesito usar posiciones de lista por lista de hijos
public void removeExternalNode (Position<E> p) {
   if( isEmpty() ) throw new InvalidPositionException( “Arbol vacío” );
   TNodo<E> n = checkPosition( p );
   if( !n.getHijos().isEmpty() ) throw new InvalidPositionException( “p no es hoja” );
   if ( p == root() ) { raiz = null; size = 0; n.setElemento(null); return; } // Parche por caso no considerado
   TNodo<E> padre = n.getParent(); // Obtengo el padre de n
   PositionList<TNodo<E>> hijosPadre = padre.getHijos(); // Obtengo los hijos del padre de n
   boolean encontre = false; // Hago una búsqueda lineal de n en la lista de hijos del padre de   n
   tdalista.Position<TNodo<E>> pp = null; // va a almacenar la posición de n en la lista de hijos de su padre
   Iterable<tdalista.Position<TNodo<E>>> posiciones = hijosPadre.positions();
   Iterator<tdalista.Position<TNodo<E>>> it = posiciones.iterator();
   while ( it.hasNext() && !encontre ) { // Para cada posición pp de la lista de hijos del padre de n
      pp = it.next(); // Pido la siguiente posición de nodo
      if ( pp.element() == n ) encontre = true; // Testeo si pp es la posición de n
   }
   if( !encontre ) // No encontré a n entre los hijos de su padre: entro en pánico, no hago nada y aviso
      throw new InvalidPositionException( “p no aparece en la lista de hijos de su padre: ¡no eliminé!” );
   hijosPadre.remove( pp ); // La búsqueda fue exitosa: elimino a n de la lista de hijos de su padre
   n.setElemento(null); // Puedo setear el rótulo en null para ayudar a checkPosition
   n.setPadre(null);
   size--; // Decremento el tamaño del árbol
}
```

$T_{removeExternalNode}(n) \rightarrow O(\text{cantidad de hijos del padre de p})$

$\leq$

$\leq O(\text{grado del árbol})$

children(v)

Estrategia:

- Tengo que devolver una lista con los hijos de un nodo v
  - Creo una copia de la lista para proteger la estructura del árbol → lista de salida
  - Itero sobre los hijos de v
  - “Encolo” en la lista de salida cada uno de los hijos

children(v)

```
public Iterable<Position<E>> children( Position<E> v ) throws InvalidPositionException {
    TNodo<E> p = checkPosition(v);
    PositionList<Position<E>> lista = new ListaDoblementeEnlazada<TNodo<E>>();
    for( TNodo<E> n : p.getHijos() )
        lista.addLast(n);
    return lista;
}
```

$T_{children}(n) \rightarrow O(\text{cantidad de hijos de v}) \leq$
$\leq O(\text{grado del árbol})$

positions()

Estrategia:

- Tengo que devolver un iterador de posiciones con los nodos del árbol
  - Recorro el árbol (preorden)
  - Agregando al final de una lista los nodos del árbol

positions()

```
public Iterable<Position<E>> positions() {
        PositionList<Position<E>> l = new ListaDoblementeEnzalada<Position<E>>();
        // Si el árbol no está vacío, hago un recorrido preorden desde la raíz:
        if( !isEmpty() )
                pre(raiz, l);
        return l;
}
```

positions()

```
public Iterable<Position<E>> positions() {
        PositionList<Position<E>> l = new ListaDoblementeEnzalada<Position<E>>();
        // Si el árbol no está vacío, hago un recorrido preorden desde la raíz:
        if( !isEmpty() )
                pre(raiz, l);
        return l;
}
private void pre(TNodo<E> v, PositionList<Position<E>> l) {
        l.addLast( v ); // La visita de v consiste de “encolar” v en l
        for( TNodo<E> h : v.getHijos()) // para cada hijo h de v hacer
                pre( h, l ); // preorden de h
}
```

$T_{positions}(n) \rightarrow O(n)$

¿Podría hacer que ***pre*** sea una función en lugar de un procedimiento?

¿Podría hacer otro tipo de recorrido (postorden, niveles, etc)?